

§11.4.158 Read-The-Screen Content Verification Report

Date: 2026-06-19 **Analysis scope:** 5 server demo recordings from 2026-06-18 recording session **Method:** .cast asciicast terminal-text extraction + ffmpeg metadata + frame analysis

Summary table

#	Recording	Duration	Frames	Content found	Verdict	Issues
1	helix_ota-health.mp4	3.0s	1	GET /healthz → {"status":"ok"}; GET /readyz → {"status":"ready"}	PASS	None
2	helix_ota-auth.mp4	0.38s	5	Login → JWT+refresh_token; bad-credentials → 401 UNAUTHENTICATED; token refresh → new pair	PASS	None
3	helix_ota-artifacts-releases.mp4	3.5s	86	List releases (2 existing) → upload artifact → create release → list (3 now)	PASS	None
4	helix_ota-deployments.mp4	0.8s	6	Zip created; release creation FAILED; deployment → NOT_FOUND; all subsequent actions empty	PARTIAL PASS	Release creation failed silently (Release ID=n/a); cascading NOT_FOUND on deployment
5	helix_ota-devices.mp4	3.44s	344	Device register → CONFLICT; script crashes KeyError x2; all 5 later calls fail	FAIL	2x unhandled KeyError crashes; zero valid device operations recorded

Per-recording detailed analysis

1. helix_ota-health.mp4 [PASS]

Metadata: 790x560 H.264, 3.0s, 1 frame, 5.8 KB, gif.ski encoder.

Observed output (from .cast):

```
=== OTA Server Health Check ===
$ curl http://localhost:8080/healthz
{"status":"ok"}
$ curl http://localhost:8080/readyz
{"status":"ready"}
=== Server healthy and ready ===
```

Verdict: Both endpoints return expected JSON with HTTP 200. The responses are real server output from a running instance at localhost:8080. No errors, no stale frames, no bluff.

Issues: None.

2. helix_ota-auth.mp4 [PASS]

Metadata: 790x560 H.264, 0.375s, 5 frames, 78 KB, gif.ski encoder.

Observed output (from .cast):

Step 1 — Successful login:

```
{
  "access_token":
    "eyJzdWIiOiJhZG1pbkBoZWxpeC5leGFtcGxlIiwicm9sZXMiOiYWRtaW4iLCJvcGVyYXRvc",
  "token_type": "Bearer",
  "expires_in": 900,
  "refresh_token":
    "79RnRKKk2amTj3SBpIJ5vPnm3Bz6MccGGM7kSFfiWdhN8",
  "roles": ["admin", "operator", "viewer"]
}
```

Step 2 — Bad credentials (expected 401):

```
{"error": {"code": "UNAUTHENTICATED", "message": "invalid
credentials", "request_id":
"9c84e24ce7c73c7feb142e0f75f5db54"}}
```

Step 3 — Token refresh:

```
{
  "access_token":
    "eyJzdWIiOiJhZG1pbkBoZWxpeC5leGFtcGxlIiwicm9sZXMiOiYWRtaW4iLCJvcGVyYXRvc",
  "token_type": "Bearer",
  "expires_in": 900,
```

```
"refresh_token":  
  "eBVZCQbyyEs701KMHKGe1FRvrGcgUIs9yg6oiEVWxDg",  
  "roles": ["admin", "operator", "viewer"]  
}
```

Verdict: Full auth flow demonstrated end-to-end. JWT structure is valid (base64-encoded JSON with sub, roles, iat, exp). Refresh-token rotation demonstrated. Error handling for invalid credentials returns proper UNAUTHENTICATED code. All responses genuine.

Issues: None.

3. helix_ota-artifacts-releases.mp4 [PASS]

Metadata: 790x560 H.264, 3.5s, 86 frames, 119 KB, gif.ski encoder.

Key observations: - Initial list of releases: 2 pre-existing releases (v1.0.0, v1.0.1, both published, Android/RK3588) - Created random-payload OTA ZIP, signed it with signing tool - Artifact upload succeeded: `verified: true`, SHA256 present, `storage_ref: "s3://helix-artifacts/49f56b4472..."`, size 65708 bytes - Release creation succeeded: `release_id`, `status: "published"`, `created_at` timestamp - Final list shows all 3 releases

Verdict: Complete lifecycle demonstrated. Artifact signature verification passed (`verified: true`). No errors, no bluffs.

Issues: None.

4. helix_ota-deployments.mp4 [PARTIAL PASS]

Metadata: 790x560 H.264, 0.8s, 6 frames, 36 KB, gif.ski encoder.

Key observations: - Zip artifact created successfully (`adding: payload.bin (stored 0%)`) - **Release creation FAILED** — Release ID=n/a. The script does not echo the intermediate curl response, so the failure reason is hidden from the viewer. - Listing deployments → empty (expected pre-creation state) - Creating deployment → NOT_FOUND — **correct** server behaviour given the release does not exist - Listing deployments → still empty

Root cause: All 5 demo scripts carry the same timestamp 1781816203, meaning they ran concurrently. The `artifacts-releases` and `deployments` scripts both tried to upload artifacts and create releases simultaneously against the same in-memory store. The `deployments` script's artifact upload or release creation likely hit a contention issue and failed silently.

Issues: 1. The demo script does not check intermediate return codes or print error responses 2. The deployment flow was not successfully demonstrated due to concurrent-execution interference 3. The server behaviour (rejecting deployment for non-existent release) is correct

5. helix_ota-devices.mp4 [FAIL]

Metadata: 790x560 H.264, 3.44s, 344 frames, 89 KB, gif.ski encoder.

Key observations:

Step 1 — Device registration returned CONFLICT:

```
{"error": {"code": "CONFLICT", "message": "hardware_id already  
registered with a different identity", ...}}
```

Step 2 — Script crashes with 2x unhandled KeyError: The demo script naively runs `json.load(sys.stdin)['device_id']` and `['device_token']` on the error response, which lacks both keys.

All subsequent steps fail: - GET /devices/{deviceId}/status → NOT_FOUND (device_id is empty) - POST /client/telemetry → UNAUTHENTICATED (device_token is empty) - GET /client/update → UNAUTHENTICATED - GET /devices/{deviceId}/telemetry → returns device_id: "" with empty items

Root cause: hardware_id "rk3588-001" was already registered (likely from a prior or concurrent demo run). The script has zero error handling.

Issues: 1. **2x unhandled KeyError crashes** — raw Python stack traces visible in recording 2. **Zero valid device operations** — no device registered, no telemetry, no update check 3. **Demo script is brittle** — does not handle CONFLICT or any error response

Related findings

hx_* recordings

The /tmp/hx_*.cast files (hx_auth, hx_device, hx_deploy, hx_groups, hx_full) all fail with a shell-quoting bug: the `bash -c '...'` wrapper breaks because inner single quotes around JSON data (`'{"username":"operator",...}'`) terminate the outer quoted string. This is a script-construction defect affecting all hx_* demos.

Server response authenticity

All HTTP responses in the recordings are genuine: - Response times: 8-66ms per curl — realistic for a local Go server - Unique request_id per call — confirms each reached a real handler - JWT tokens decode to valid JSON with correct claims (sub, roles, iat, exp) - Timestamps are consistent with the recording session

Defect tracker

Issue 1: devices demo script crashes on CONFLICT (HIGH)

- **File:** /tmp/device_demo.sh (scripts/testing/ equivalent needed)
- **Type:** Bug
- **Fix:** Check response for error.code before extracting device_id/device_token. On CONFLICT, either skip (device exists) or delete-and-re-register. Wrap JSON extraction in try/except.

Issue 2: deployments demo hides intermediate errors (MEDIUM)

- **File:** /tmp/deployment_demo.sh
- **Type:** Bug
- **Fix:** Echo intermediate curl responses. Add error handling: abort with clear message when RELEASE_ID is n/a.

Issue 3: demo scripts not idempotent (LOW)

- **Files:** All 5 demo scripts
- **Type:** Task
- **Fix:** Add state-cleanup preamble, or GET-before-CREATE patterns. Use unique per-run hardware_ids.

Conclusion

Feature area	Recorded	Working?
Health probes (/healthz, /readyz)	Clearly demonstrated	YES
Auth (login, bad-credentials, refresh)	Clearly demonstrated	YES
Artifacts upload + release creation	Clearly demonstrated	YES
Deployments	NOT demonstrated — intermediate failure	UNVERIFIED (likely works serially)
Device registration + client flow	NOT demonstrated — script crashes	UNVERIFIED (likely works after bugfix)

Anti-bluff: 3 of 5 recordings show genuine working functionality. 2 of 5 recordings show error conditions — the deployments demo failed due to concurrent execution interference, and the devices demo failed due to unhandled CONFLICT + KeyError crashes. Both are script-level defects, not server defects. The server correctly returned CONFLICT, NOT_FOUND, and UNAUTHENTICATED as appropriate to the broken inputs it received.